

N100 FINANCIAL INTELLIGENCE PLATFORM

Analyst & Operations Guide

Comprehensive User Manual & Technical Operation Reference

Document Details

Version: 1.0 (Production Release)

Sprint: Sprint 6 — Performance & Integration

Date: August 2026

Target Universe: Nifty 100 Companies (92 Validated Constituents)

Core Technology Stack

- **Language:** Python 3.11 / 3.12 (Standard Library)
- **Data Processing:** Pandas, NumPy, OpenPyXL, PyYAML
- **Relational Engine:** SQLite 3 (WAL Mode, Transactional Integrity)
- **Application Hosting:** FastAPI / Uvicorn REST Backend, Streamlit UI Dashboard
- **Report Generator:** ReportLab PDF Compilation Engine
- **Verification:** Pytest Test Suite (211 Tests, 100% Pass)

1. Platform Overview

The Nifty 100 Financial Intelligence Platform is a local-first financial research and analytics engine. It is designed to automate the process of ingesting raw corporate financial spreadsheets, validating their content against a set of strict schemas and financial integrity rules, storing the standardized data in a structured relational database, and exposing the resulting intelligence via programmatic REST API endpoints and an interactive web-based dashboard.

Core Architecture Pipeline

The system coordinates data and control flows through seven primary lifecycle stages:

Stage	Component	Role & Operation
1. Ingestion	ETL Excel Loader	Parses incoming corporate filing spreadsheets; auto-detects row/column offsets.
2. Validation	DQ Rules Engine	Applies 16 structural and consistency checks; isolates errors in dedicated logs.
3. Storage	Indexed SQLite DB	Stores schema-validated data in a normalized database configured in WAL mode.
4. Analytics	KPI & CAGR Engines	Computes 50+ profitability, liquidity, and growth KPIs with division-by-zero guards.
5. PDF Engine	ReportLab Compiler	Compiles 2-page company tearsheets and sector summary booklets automatically.
6. REST Layer	FastAPI Server	Serves 24 versioned JSON routes with autogenerated interactive Swagger docs.
7. Interface	Streamlit Dashboard	Renders interactive charts, watchlists, and sliding-scale screening presets.

By enforcing strict typing, mathematical validation, and relational constraints before presentation, the platform ensures that financial analysts examine highly accurate datasets free from sheet offset errors or missing balance-sheet values.

2. Installation & Environment Setup

The platform is designed to run locally, eliminating external cloud dependency and ensuring maximum data privacy. Follow these steps to set up the runtime environment on your local workstation.

1. Codebase Retrieval

Clone the version-controlled repository to your local directory:

```
git clone https://github.com/krishnavasnani07/n100-financial-intelligence-platform.git
cd n100-financial-intelligence-platform
```

2. Virtual Environment Configuration

Create a sandboxed virtual environment using Python 3.11 or 3.12, then activate it:

```
# For Windows Workstations (PowerShell)
python -m venv .venv
.venv\Scripts\activate

# For macOS and Linux Terminals
python -m venv .venv
source .venv/bin/activate
```

3. Dependency Installation

Install the minimal required package dependencies using pip:

```
pip install -r requirements.txt
```

4. Environment Settings (.env)

The system reads configurations from a local environment file. Copy the example configuration template `.env.example` to `.env` in the root folder, and adjust values as needed:

```
ENV=development
DEBUG=True
DB_PATH=db/nifty100.db
LOG_LEVEL=INFO
LOG_FILE=logs/app.log
```

3. Running the ETL & Analytics Pipeline

The ETL (Extract, Transform, Load) pipeline processes raw corporate spreadsheets, standardizes names, runs data quality checks, inserts valid records into the SQLite database, and computes financial ratios.

1. Pipeline Execution

Run the master ETL pipeline using the following command in the project root:

```
# Set PYTHONPATH to the current directory to enable internal module routing
$env:PYTHONPATH="." # Windows PowerShell
export PYTHONPATH="." # macOS/Linux

# Run the loader pipeline
python main.py
```

2. Pipeline Stages & Operations

When `main.py` is executed, the following actions occur in sequence:

- A. Parsing & Coordinate Alignment:** The *ExcelLoader* parser scans sheets in *data/raw/*. It automatically identifies structural row and column offsets by searching for anchor keywords (e.g., 'Revenue', 'Equity').
- B. Name & Ticker Standardization:** Tickers are cleaned and matched against a master metadata list (e.g. standardizing NSE formats like *TCS.NS* to *TCS*) to prevent duplicate profiles.
- C. Data Quality Evaluation:** The raw dataset is evaluated against 16 structural checks. If a sheet contains critical errors (e.g., \$Assets eq Liabilities\$), it is rejected, and an error report is exported to *output/parse_failures.csv*.
- D. Relational Storage:** Schema-compliant financial data is loaded into the SQLite database under a single database transaction. If any load operation fails, the transaction auto-rolls back to prevent database corruption.
- E. Analytical Ratios:** After ingestion, the system runs mathematical ratio engines to calculate DuPont profitability margins, historical CAGR growth rates, and cash flow indices. The processed ratios are saved in the *financial_ratios* table.

4. Running the Streamlit Dashboard

The interactive analytics dashboard provides a clean user interface for equity research. It queries the cached SQLite database and displays data using responsive Plotly charts.

1. Startup Command

Launch the Streamlit web application from your terminal:

```
streamlit run app.py --server.port 8501
```

Once started, open your web browser and navigate to: **http://localhost:8501**

2. Dashboard Screens Directory

Navigate between the 8 specialized research views via the sidebar menu:

Screen Page	Key Features	Primary Usage
1. Executive Home	Treemaps, sector distribution charts, and composite rankers.	Overview of market performance and quality distributions.
2. Company Profile	Margin charts, DuPont breakdown, leverage tables, and watchlists.	Deep-dive research into a specific company's financial sheets.
3. Investment Screener	10 interactive sliders, 6 predefined strategy presets, CSV export.	Filtering and identifying stocks matching target parameters.
4. Peer Comparison	Normalized multi-variable radar charts, relative valuations.	Comparing a stock directly with its closest sector peers.
5. Trend Analysis	10-year historical metrics, YoY changes, CAGR curves.	Tracking a firm's long-term sales, earnings, and cash trends.
6. Sector Analytics	Interactive bubble plots (Revenue vs. ROE), index weights.	Analyzing industry structures and finding sector leaders.
7. Capital Allocation	Treemaps grouping allocation profiles (e.g. Dividend Leaders).	Evaluating how cash flow is reinvested or returned to owners.
8. Reports Browser	Search overlay, PDF viewer, annual filings directory.	Reviewing local annual report source documents.

5. Using the Quality Screener

The Investment Screener helps analysts identify and rank companies that meet custom financial thresholds. It features interactive sliders and predefined filters.

1. Preset Filtering Strategies

The screener includes 6 predefined, sector-adjusted templates that populate the sliding filters:

Strategy Preset	Key Parameters Implemented	Target Category
Quality Compounder	ROE > 18%, ROCE > 18%, Debt/Equity < 0.5x, positive FCF.	High return, low leverage firms.
Value Pick	P/E < Sector Median, FCF Yield > 5%, ROE > 12%.	Undervalued companies with cash.
Dividend Champion	Dividend Yield > 3%, Payout Ratio < 75%, positive cash flows.	Stable, high-yield cash generators.
Growth Accelerator	Revenue CAGR (3Y) > 15%, PAT CAGR > 15%, OPM > 10%.	Rapidly expanding businesses.
Debt-Free Blue Chip	Debt/Equity = 0.0x, Market Cap > Large Cap threshold.	Ultra-stable, conservative firms.
Turnaround Watch	Improving OPM margins, positive FCF, turning profitable.	Potential recovery candidates.

2. Operational Workflow Step-by-Step

- Step 1: Open the Screener.** Click 'Investment Screener' in the Streamlit sidebar.
- Step 2: Choose a Strategy or Adjust Sliders.** Select one of the presets to automatically adjust the filters, or manually drag the 10 sliding parameters (ROE, ROCE, PE, etc.).
- Step 3: Filter by Industry.** Use the sector checklist multi-select box to isolate companies in a specific sector (e.g. *Information Technology*).
- Step 4: Review Ranked Outputs.** The platform ranks qualifying companies by their quality scores and displays their metrics in a responsive data table.
- Step 5: Export Data.** Click the 'Export to CSV' or 'Export Excel Report' buttons to download the screen's output for external models.

6. Company Profile & Tearsheets

The Company Profile view provides detailed financial ratios, margin trends, DuPont parameters, and custom visualizations for individual stocks. It also supports compiling PDF tearsheets.

1. Research Navigation

Open 'Company Profile' in the sidebar navigation. Type or select a ticker (e.g., TCS) in the autocomplete search box. The page will fetch metrics from the SQLite database.

2. Visual Analytics Sections

- **Margins Breakdown:** Compares gross profit, operating profit, and net profit margins over time.
- **DuPont Decomposition:** Breaks down return on equity (ROE) into profit margin, asset turnover, and leverage multiplier.
- **Balance Sheet Health:** Displays leverage and liquidity trends, highlighting debt levels and coverage metrics.

3. PDF Tearsheet Generation

To compile a publication-quality PDF report, click the '**Generate PDF Tearsheet**' button. The PDF generation pipeline will build a structured 2-page report for the selected stock. Generated PDFs are saved in: reports/tearsheets/.

```
reports/tearsheets/  
■■■ TCS_tearsheet.pdf
```

4. Batch PDF Generation Pipeline

You can run the batch generation script to generate tearsheets for all 92 validated companies and sector summaries in one run. Execute this script from the project root:

```
python src/reports/batch_generator.py
```

7. FastAPI REST API Guide

FastAPI exposes the platform's analytical databases through versioned REST endpoints. This allows external scripts, Python scripts, and third-party tools to fetch data from the SQLite store.

1. Starting the API Server

Launch the FastAPI backend server using Uvicorn:

```
uvicorn src.api.main:app --host 127.0.0.1 --port 8000 --reload
```

2. Interactive Documentation Documentation Links

Once started, access interactive UI pages and documentation files:

- **Swagger Interactive UI:** <http://localhost:8000/docs>
- **ReDoc Technical View:** <http://localhost:8000/redoc>
- **OpenAPI JSON Schema File:** [docs/openapi.json](#)

3. Core Endpoint References

The API includes 24 routes. The primary endpoints are:

Method	Endpoint Path	Parameters	Description
GET	/api/v1/health	None	Database health and table row checks.
GET	/api/v1/companies	sector (optional)	List of all tickers in database.
GET	/api/v1/companies/{ticker}	ticker (in path)	Full profile details and sector metrics.
GET	/api/v1/companies/{ticker}/ratios	ticker (in path)	10-year historical ratios and margins.
GET	/api/v1/screener	min_roe, max_de, etc.	Filters stocks by parameters.
GET	/api/v1/sectors	None	Aggregated metrics across sectors.
GET	/api/v1/peers/{group}	group (in path)	Peer ranks and sector medians.
GET	/api/v1/valuation	None	Undervalued and premium classifications.

8. API Examples

Below are examples of API requests and responses. Use these templates to configure integrations with external tools or scripting environments.

1. Health Status Request

```
curl -X GET "http://localhost:8000/api/v1/health"
```

Expected JSON Response:

```
{ "status": "healthy", "database_connected": true, "companies_count": 92, "financial_ratios_count": 1104 }
```

2. Company Ratio Profile Request

```
curl -X GET "http://localhost:8000/api/v1/companies/TCS/ratios"
```

Expected JSON Response Structure:

```
{ "ticker": "TCS", "ratios": [ { "year": 2025, "roe_pct": 48.2, "roce_pct": 56.4, "debt_to_equity": 0.02, "operating_margin_pct": 26.5, "net_margin_pct": 19.8 } ] }
```

3. Stock Screener Request

```
curl -G "http://localhost:8000/api/v1/screener" \ --data-urlencode "min_roe=18" \ --data-urlencode "max_de=0.5"
```

Expected JSON Response Structure:

```
{ "count": 28, "results": [ { "ticker": "INFY", "name": "Infosys Limited", "sector": "Information Technology", "roe_pct": 29.8, "debt_to_equity": 0.08 } ] }
```

9. Troubleshooting operations

Use this matrix to diagnose and resolve common errors and operational issues when running the platform.

Observed Problem	Likely Root Cause	Recommended Action
FastAPI won't start	Port 8000 is occupied by another local service.	Identify and stop the process using port 8000, or run the server on a different port: <code>uvicorn src.api.main:app --port 8001</code> .
Streamlit won't start	The virtual environment is not activated, or dependencies are missing.	Run the activation command (e.g., <code>.venv\Scripts\activate</code>) and re-run <code>pip install -r requirements.txt</code> .
Database error / empty tables	The database path in <code>.env</code> is incorrect, or the database hasn't been created.	Verify that <code>DB_PATH=db/nifty100.db</code> in your <code>.env</code> file, and run the ETL script: <code>python main.py</code> .
Database lock / slow queries	Parallel writes locked the database file.	Ensure SQLite is running in WAL mode (run <code>PRAGMA journal_mode=WAL;</code>). Avoid concurrent write operations.
Screener yields empty results	Filters are too restrictive.	Widen your parameters (e.g., reduce the minimum ROE requirement or increase the maximum Debt/Equity threshold).
PDF Tearsheet fails to compile	ReportLab is missing, or the output directories do not exist.	Create the output folders: <code>mkdir reports/tearsheets</code> . Check that ReportLab is installed.
API returns 404 Not Found	The ticker format is invalid, or the sector name is misspelled.	Confirm the ticker is in the database using <code>/api/v1/companies</code> . Check for capitalization.

10. Developer / Technical Reference

This reference section outlines the codebase structure, database schema, and test suite conventions.

1. Codebase Architecture Layout

- ``src/etl/``: Extraction scripts that parse Excel raw data templates.
- ``src/validation/``: Quality assurance rules verifying structural formatting and consistency.
- ``src/database/``: Relational database connection pools, setup schemas, and loading scripts.
- ``src/analytics/``: Core mathematical engines for ratio and CAGR calculations.
- ``src/api/``: FastAPI routers, request schemas, and logging middleware.
- ``src/reports/``: ReportLab canvas configurations for tearsheet and sector digest compiles.
- ``tests/``: Pytest unit, API, and system integration test suite.

2. Database Schema Indexing

To support fast, sub-millisecond query responses, the relational SQLite database includes the following indexes:

- **Primary Keys**: Auto-indexed tables for fast query lookups by ID.
- ``idx_ratios_company_year``: Composite index on ``(company_id, year)`` in the ``financial_ratios`` table.
- ``idx_pl_company_year``: Composite index on ``(company_id, year)`` in the P&L; table.
- ``idx_sectors_company_id``: Index mapping company IDs directly to their sector classifications.

3. Running the Test Suite

The platform includes 211 tests. Verify the code quality by running the test suite locally:

```
# Set path environment variable
$env:PYTHONPATH="."

# Run all tests
pytest tests/ -v

# Export test results to an HTML report file
pytest tests/ --html=reports/pytest_report.html
```